

enforcing them throughout long-horizon development.

As illustrated in Figure 1, existing agents face two closely related challenges:

- **Unreliable functional-chain design.** Existing agents typically design features through free-form reasoning, which can deviate from repository architecture constraints and overlook the call and dependency relations among code entities. Consequently, they may derive functional chains that are incomplete or inconsistent with the repository.
- **Weak long-horizon enforcement.** Functional designs are typically represented as free-form textual plans, which are difficult to verify and enforce during iterative exploration, editing, and repair. As a result, the implementation may gradually diverge from the intended functional chain, leading to inconsistencies between design and realization.

To address these challenges, we propose CODESPEC, a dual executable specification method for agentic long-horizon feature development. It constructs repository-grounded functional-chains for feature sub-requirements and compiles each chain into complementary architecture and behavior executable specifications. CODESPEC consists of two key components:

1) Evidence-Grounded Functional Chain derives traceable feature realization paths by identifying relevant functional units and connecting them with paired requirement semantics and repository as evidence, such as design patterns, call relations, and dependencies. This reduces free-form architectural reasoning and grounds each transition in the existing repository to improve the design reliability.

2) Dual Executable Specification Compilation converts each functional chain into two executable forms: an architecture specification that represents its architecture realization and a behavior specification that represents its expected execution. They complementarily verify whether the chain is complete and whether it behaves correctly, thereby preserving design–implementation consistency throughout long-horizon development.

Experiments on FeatureBench (Zhou et al. 2026) show that CODESPEC achieves 70.7%, 55.0%, and 49.9% %Passed on the Lite, Fast, and Full splits, consistently outperforming coding-agent and architecture-planning baselines under the same DeepSeek-V4-Pro backbone. Results with GPT-5.4-mini and NL2Repo-Bench (Ding et al. 2026) which evaluates greenfield repository development from natural-language requirements further demonstrate robustness across models and generalization to greenfield repository generation. Ablation and further analyses confirm the effectiveness of evidence-grounded functional chains and the complementary executable specifications, particularly on complex and long-horizon tasks, with moderate additional cost.

Our main contributions are summarized as follows:

- We propose evidence-grounded functional-chain reasoning that pairs task semantics with repository architecture evidence at each transition, enabling agents to derive reliable cross-component realization paths from feature entry points to observable behaviors.
- We introduce dual executable specifications that represent the architecture realization and expected execution

of each functional chain, jointly verifying whether the chain is complete and behaves correctly while maintaining design–implementation consistency throughout long-horizon development.

- We conduct extensive experiments on FeatureBench and NL2Repo with different backbone models, demonstrating the effectiveness, robustness, generalizability, and favorable quality–cost trade-off of CODESPEC for long-horizon feature development.

Related Work

Repository-level Feature Development Benchmarks

Code generation research has evolved from standalone function synthesis (Chen 2021; Austin et al. 2021) to repository-level software development, where models need to understand project context and generate patches in real codebases. Benchmarks such as DevEval (Li et al. 2024) and SWE-bench (Jimenez et al. 2024) evaluate such repository-level capabilities through realistic software engineering tasks. More recent feature-level benchmarks, including FEA-Bench (Li et al. 2025), and FeatureBench (Zhou et al. 2026), further focus on practical feature development, where a high-level instruction may require coordinated changes and generation across multiple functions, files, classes, and interfaces. These tasks usually involve a full development process of design, implementation, testing, execution, and repair, making one-shot generation insufficient and motivating coding agents that interact with repositories and development environments over multiple steps.

LLM-based Coding Agents

LLM-based coding agents have become a popular paradigm for repository-level software development (Wang et al. 2026, 2025b,a). Unlike one-shot generation methods, they can autonomously invoke tools, inspect files, text plan, edit programs, and iteratively repair failures. Representative coding agents and assistants include Claude Code (Anthropic 2026), Codex (OpenAI 2025), SWE-agent (Yang et al. 2024), and OpenHands (Wang et al. 2025c). In addition, development-oriented agent frameworks such as MetaGPT (Hong et al. 2024), ChatDev (Qian et al. 2024) and RTADev (Liu et al. 2025) organize software development into structured stages and use textual architecture designs to coordinate software development. Despite their progress, existing agents still mainly maintain design information as free-form textual plans, which makes it difficult to explicitly preserve functional chains and architecture constraints during long-horizon feature development.

Method

Task Definition

In this section, we define the **feature development** task. Given a natural-language feature instruction I and an existing repository R , an LLM-based coding agent generates a patch P to get the updated repository R' which preserving existing functionality and satisfying the instruction I :

$$P = \text{LLM}(I, R), \quad R' = R \oplus P. \quad (1)$$

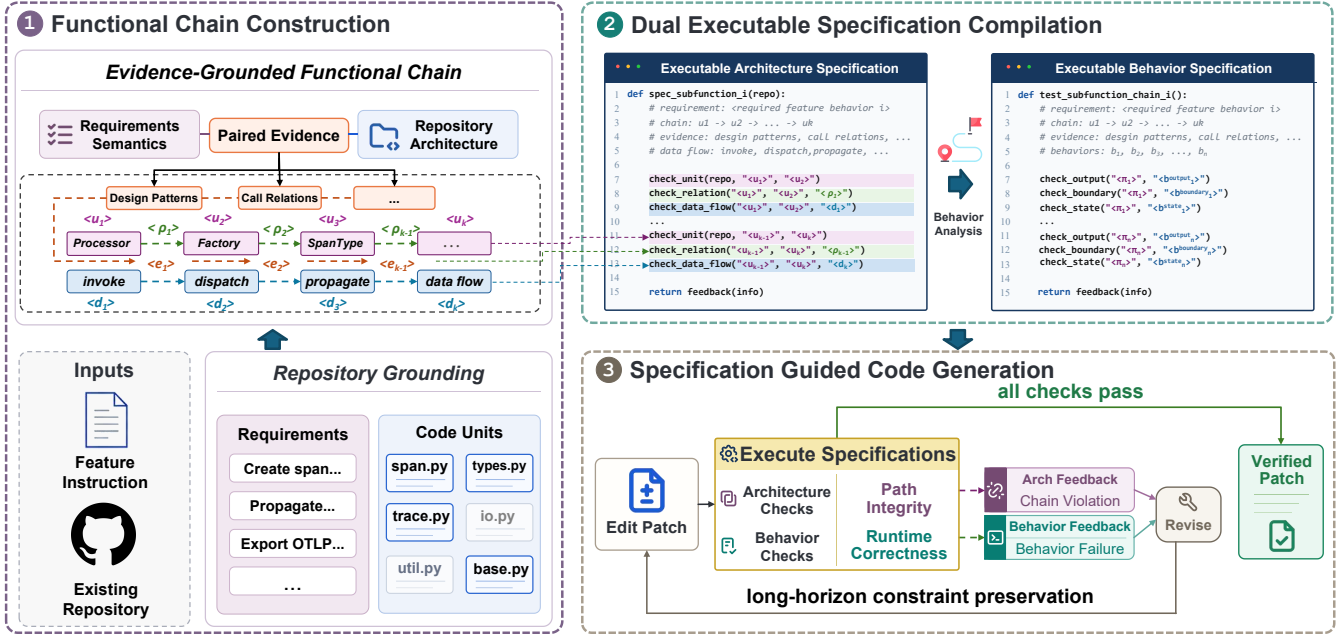


Figure 2: The overview of CODESPEC.

Overview

As shown in Figure 2, CODESPEC constructs reliable functional chains using evidence that pairs sub-requirement semantics with the corresponding repository architecture. It then compiles each chain into dual executable specifications: an architecture specification representing its structural realization and a behavior specification representing its expected execution. These specifications complementarily verify whether the chain is complete and behaves correctly, thereby maintaining design–implementation consistency throughout long-horizon feature development.

Evidence-Grounded Functional Chain

A feature is realized through coordinated interactions among multiple repository components rather than isolated code modifications. However, feature instructions usually describe desired behaviors without specifying which components should participate or how they should interact. To bridge this gap, CODESPEC constructs an evidence-grounded functional chain for each sub-function by pairing requirement semantics with repository architecture evidence at every reasoning step.

Given a feature instruction I , the agent first decomposes it into a set of sub-requirements:

$$\text{LLM}(I) = \{r_1, r_2, \dots, r_m\}. \quad (2)$$

Each requirement r_i represents a specific behavior that the feature should achieve. For each requirement, the agent identifies candidate implementation units that need to be modified or introduced:

$$U_{r_i} = \{u_1, u_2, \dots, u_n\}. \quad (3)$$

These units provide candidate building blocks, but selecting relevant units alone does not yield a valid feature design; the key is determining how they should be connected to realize each obligation. CODESPEC therefore prompts the LLM to perform evidence-driven chain reasoning by pairing requirement evidence, which explains why a transition is needed from requirements, with repository evidence, which shows how it is supported by existing design patterns, call relations, or repository architectures. The resulting functional chain is an ordered sequence in which each adjacent pair of units u_i^k and u_{i+1}^k is linked according to its supporting evidence e_i^k :

$$c_i = \{u_i^1, e_i^1, u_i^2, \dots, u_i^k, e_i^k, u_i^{k+1}\}. \quad (4)$$

Here, u_i^k denotes a functional unit involved in realizing requirements r_i , and e_i^k denotes the evidence supporting the transition from u_i^k to u_{i+1}^k . Each evidence e_i^k pairs requirement semantics with repository evidence, ensuring that adjacent units are connected through traceable, repository-supported reasoning rather than free code relations.

Dual Executable Specification Compilation

The constructed functional chains explicitly describe how individual feature requirement should be realized across repository components. However, retaining these chains as textual design artifacts is unsuitable for long-horizon development, because agents may gradually overlook pre-designed key information during long-term interactions. Therefore, we compile each functional chain into two complementary executable specifications: the architecture specification that verifies path integrity and the behavior specification that verifies runtime correctness along the path.

Executable Architecture Specification Given a sub-requirement r_i and its functional chain c_i , CODESPEC instan-

tiates an executable architecture specification s_i^{arch} using the template τ_{arch} shown in Figure 2:

$$s_i^{\text{arch}} = \text{LLM}_{\text{arch}}(R, r_i, c_i; \tau_{\text{arch}}). \quad (5)$$

Inspired by architecture conformance checking and program analysis (Pruijt et al. 2017; Murphy, Notkin, and Sullivan 2001; Kildall 1973), we operationalize path integrity using three representative dimensions: functional units, architectural relations, and data flows. The template maps the units, relations, and data flows in c_i into three types of checks:

$$\begin{aligned} s_i^{\text{arch}} = & \left\{ \text{CheckUnit}(u_i^j) \right\}_{j=1}^{k_i} \\ & \cup \left\{ \text{CheckRelation}(u_i^j, u_i^{j+1}, \rho_i^j) \right\}_{j=1}^{k_i-1} \\ & \cup \left\{ \text{CheckDataFlow}(u_i^j, u_i^{j+1}, d_i^j) \right\}_{j=1}^{k_i-1}. \end{aligned} \quad (6)$$

Here, ρ_i^j and d_i^j are jointly inferred from the requirement and repository evidence in e_i^j , representing the intended architectural relation and data-flow operation between u_i^j and u_i^{j+1} , respectively. **CHECKUNIT** verifies that each required unit exists, **CHECKRELATION** checks whether adjacent units preserve the intended relation ρ_i^j , and **CHECKDATAFLOW** verifies the required data state d_i^j between them.

During implementation, the specification is executed against the updated repository. Missing units, broken relations, or interrupted data flows are reported as localized architecture violations. In this way, executable architecture specifications preserve the path integrity of each functional chain.

Executable Behavior Specification Given the sub-requirement r_i , its functional chain c_i , and the corresponding architecture specification s_i^{arch} , **CODESPEC** jointly analyzes the intended path structure and architectural constraints to identify behavior-observation subpaths, and instantiates an executable behavior specification using the template τ_{beh} shown in Figure 2:

$$s_i^{\text{beh}} = \text{LLM}_{\text{beh}}(r_i, c_i, s_i^{\text{arch}}; \tau_{\text{beh}}). \quad (7)$$

Inspired by established property-based, category-partition, and model-based testing techniques (Claessen and Hughes 2000; Ostrand and Balcer 1988; Chow 1978), we operationalize behavioral obligations using three representative dimensions: observable outputs, boundary conditions, and state transitions. The template derives three types of behavioral checks at multiple observation scopes along the functional chain:

$$\begin{aligned} s_i^{\text{beh}} = & \{ \text{CheckOutput}(\pi_i^q, b_{i,q}^{\text{output}}), \\ & \text{CheckBoundary}(\pi_i^q, b_{i,q}^{\text{boundary}}), \\ & \text{CheckState}(\pi_i^q, b_{i,q}^{\text{state}}) \}_{q=1}^{n_i}. \end{aligned} \quad (8)$$

Here, n_i denotes the number of behavioral observation scopes derived for c_i , and $\pi_i^q \subseteq c_i$ denotes the q -th scope,

which may cover a functional unit, a transition, or a sub-path. The checks examine observable outcomes, including returned values and external effects, boundary and exceptional conditions, and state transitions at multiple positions along the functional chain. Thus, the behavior specification examines multiple intermediate and final behaviors rather than only the final output.

During implementation, violated expectations are reported as localized behavior violations. In this way, executable behavior specifications complement architecture specifications by preserving runtime correctness along each functional chain.

Specification Guided Code Generation

For all sub-requirements, the generated architecture and behavior specifications are aggregated as:

$$S^{\text{arch}} = \bigcup_{i=1}^m s_i^{\text{arch}}, \quad S^{\text{beh}} = \bigcup_{i=1}^m s_i^{\text{beh}}. \quad (9)$$

The agent implements the feature under the joint constraints of S^{arch} and S^{beh} . Architecture feedback identifies missing units, broken relations, or interrupted data flows, while behavior feedback reports incorrect runtime behaviors along the functional chains. The agent incorporates both signals into subsequent edits until all specifications pass or the interaction budget K is exhausted.

Algorithm 1: Specification Guided Code Generation

Input: Instruction I , repository R , specifications S^{arch} and S^{beh} , budget K
Output: Patch P

- 1: $P, F_{\text{arch}}, F_{\text{beh}} \leftarrow \emptyset$
- 2: $k \leftarrow 0$
- 3: **while** $k < K$ **do**
- 4: $\Delta P \leftarrow \text{LLM}_{\text{code}}(I, R, S^{\text{arch}}, S^{\text{beh}}, P, F_{\text{arch}}, F_{\text{beh}})$
- 5: $P \leftarrow P \oplus \Delta P$
- 6: $R' \leftarrow R \oplus P$
- 7: $F_{\text{arch}} \leftarrow \text{ExecArch}(R', S^{\text{arch}})$
- 8: $F_{\text{beh}} \leftarrow \text{ExecBeh}(R', S^{\text{beh}})$
- 9: **if** $\text{Pass}(F_{\text{arch}}) \wedge \text{Pass}(F_{\text{beh}})$ **then**
- 10: **break**
- 11: **end if**
- 12: $k \leftarrow \text{InteractCount}()$
- 13: **end while**
- 14: **return** P

Here, **ExecArch** verifies the integrity of the functional chains, whereas **ExecBeh** verifies their runtime behaviors. Their complementary feedback jointly preserves path integrity and behavioral correctness throughout long-horizon implementation.

Experimental Setups

Research Questions

We evaluate **CODESPEC** through the following research questions:

Agent Scaffold	Lite			Fast			Full		
	%Passed	#Resolved	Avg. Cost (\$)	%Passed	#Resolved	Avg. Cost (\$)	%Passed	#Resolved	Avg. Cost (\$)
OpenHands	60.0 $\pm$ 5.7	6	0.15	41.4 $\pm$ 3.6	10	0.13	37.1 $\pm$ 2.7	18	0.11
Claude Code	59.8 $\pm$ 5.4	6	0.11	47.0 $\pm$ 3.8	14	0.12	41.1 $\pm$ 2.8	23	0.10
Mini-SWE-Agent	62.6 $\pm$ 6.0	7	0.09	49.6 $\pm$ 4.0	24	0.09	43.4 $\pm$ 2.8	25	0.09
RTADev	65.3 $\pm$ 5.6	6	0.17	49.7 $\pm$ 3.8	15	0.16	46.1 $\pm$ 2.8	26	0.15
CODESPEC	70.7$\pm$5.1	9	0.18	55.0$\pm$3.8	17	0.19	49.9$\pm$2.8	28	0.16

Table 1: Comparison under the DeepSeek-V4-Pro (max thinking) backbone model on FeatureBench. The Lite, Fast, and Full splits contain 30, 100, and 200 tasks, respectively. Bold numbers indicate the best performance.

Agent Scaffold	%Passed	#Resolved	Avg. Cost (\$)
Mini-SWE-Agent	47.5 $\pm$ 6.0	2	0.7
OpenHands	48.1 $\pm$ 5.9	3	1.2
RTADev	45.9 $\pm$ 5.7	3	1.3
Codex	51.7 $\pm$ 6.2	4	0.7
CODESPEC	54.7$\pm$5.8	5	1.3

Table 2: Comparison of different coding agents with the same GPT-5.4-mini backbone (medium thinking) on the FeatureBench Lite split.

- **RQ1: Overall Performance.** How does CODESPEC compare with existing coding agents on repository-level feature development tasks?
- **RQ2: Ablation Study.** How does each component contribute to the CODESPEC’s performance?
- **RQ3: Executable vs. Textual Specifications.** Do executable specifications preserve design constraints more effectively than textual specifications?
- **RQ4: Greenfield Repository Development Generalization.** Can CODESPEC generalize from repository evolution to greenfield repository generation?

Benchmarks and Metrics

We evaluate CODESPEC on FeatureBench (Zhou et al. 2026), a repository-level feature development benchmark requiring coordinated modifications across mutli-interfaces. Each problem averages 4.8k words and requires about 800 lines of valid code, compared with 0.2k words and about 32 lines in SWE-Bench(Jimenez et al. 2024). It contains three splits: *Lite* (30), *Fast* (100), and *Full* (200).

Following FeatureBench, we report **%Passed**, the average proportion of fail-to-pass tests passed across tasks, and **Resolved #**, the number of tasks for which all required tests pass. The former measures partial implementation progress, while the latter measures complete task resolution. We also report the average cost metric for each task: **Avg.Cost (\$)**.

Baselines and Implementation Details

We compare CODESPEC with five representative coding-agent scaffolds:

- **OpenHands** (Wang et al. 2025c), a general-purpose agent supporting repository exploration, code editing, execution, and iterative repair.
- **Mini-SWE-Agent** (Yang et al. 2024), a lightweight agent based on a minimal repository interaction and repair loop.
- **Claude Code** (Anthropic 2026), a widely adopted industrial coding agent for autonomous repository editing, execution, and debugging.
- **Codex** (OpenAI 2025), an OpenAI coding agent supporting repository-level reasoning, tool use, code modification, and iterative refinement.
- **RTADev** (Liu et al. 2025), an architecture-first agent that uses textual architecture designs to guide development;

For controlled comparison, all methods (except for Codex) use DeepSeek-V4-Pro as the backbone model. We additionally evaluate GPT-5.4-mini to examine model-level generalization. All methods share the same repository environments, evaluation scripts, and interaction budgets. We bootstrap task-level results 10,000 times and report the resulting mean and standard deviation.

Experimental Results

RQ1: Overall Performance

As shown in Table 1, CODESPEC consistently achieves the best performance across all FeatureBench splits under the same DeepSeek-V4-Pro backbone. On the Lite split, CODESPEC reaches 70.7% %Passed and resolves 9 tasks, outperforming the strongest baseline RTADev by 5.4 percentage points in %Passed and 3 resolved tasks. Similar improvements are observed on the Fast and Full splits, where CODESPEC improves %Passed over RTADev by 5.3 and 3.8 percentage points, respectively, while achieving the highest number of resolved tasks. Compared with general-purpose coding agents such as OpenHands and Mini-SWE-Agent, CODESPEC obtains larger gains across all splits. Unlike textual plans, which remain passive context and can be overlooked during extended interactions, executable specifications continuously expose whether the intended functional chains remain complete and correct. Their lightweight feedback also helps agents localize violations without repeatedly reconsidering the entire design. These results demonstrate that explicitly modeling functional chains and enforcing them through executable specifications effectively improves repository-level feature development.

Variant	%Passed	#Resolved	Avg. Cost (\$)
<i>Specification Ablations</i>			
w/o All Specification	62.6 $\pm$ 6.0	7	0.09
w/o Behavior Specification	64.0 $\pm$ 5.2	6	0.13
w/o Architecture Specification	66.6 $\pm$ 5.7	7	0.16
<i>Evidence-Grounded Functional Chain Ablations</i>			
w/o Evidence	64.8 $\pm$ 5.7	6	0.16
CODESPEC	70.7$\pm$5.1	9	0.18

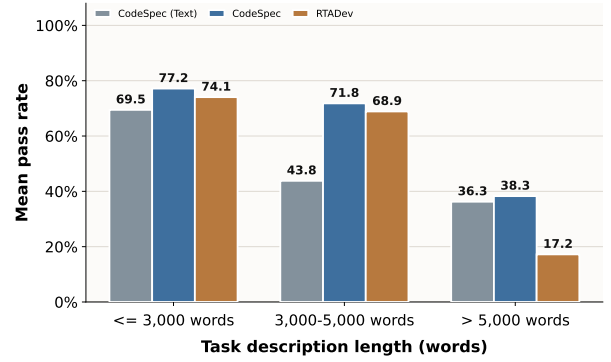
Table 3: Ablation study on the FeatureBench Lite split. All variants use DeepSeek-V4-Pro as the base model.

The additional architecture design and specification execution introduce moderate overhead. Compared with RTADev, CODESPEC increases the average cost by only \$0.01–\$0.03 per task, while consistently improving feature completion performance across different task scales. The additional interactions mainly come from executing functional-chain specifications and leveraging their violation feedback for iterative refinement, which provides more precise guidance during implementation. To further examine robustness across different backbone models, we additionally compare agent scaffolds using GPT-5.4-mini on the Lite split (Table 2). CODESPEC maintains the best performance, achieving 54.7% %Passed and resolving 5 tasks under the same backbone, even surpassing Codex, a dedicated coding agent developed by OpenAI for GPT series LLMs. These results suggest that the effectiveness of CODESPEC is robust across different backbone models rather than relying on a specific LLM.

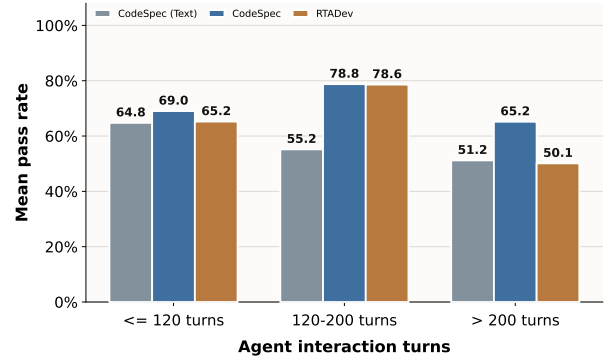
RQ2: Ablation Study

Table 3 reports the contribution of executable specifications and evidence-guided functional-chain construction. Removing all specifications decreases %Passed from 70.7% to 62.6% and reduces the number of resolved tasks from 9 to 7, confirming that persistent executable constraints are important for long-horizon feature implementation. The two specification types provide complementary benefits. Removing the behavior specification causes a substantial drop to 64.0% Passed and 6 resolved tasks, indicating that architecture connectivity alone cannot ensure correct outputs, boundary handling, or state transitions. Removing the architecture specification reduces %Passed to 66.6% and resolves 7 tasks, showing that behavior checks alone may still permit incomplete or unintended realization paths. Their combination therefore improves both path integrity and runtime correctness.

Evidence-guided functional-chain is also critical. Without explicit evidence grounding, %Passed falls from 70.7% to 64.8%, while the number of resolved tasks decreases from 9 to 6. This suggests that requirement semantics and repository evidence help identify more reliable functional units and relations, thereby reducing unsupported architectural reasoning. Although the complete method incurs the highest average cost at \$0.18 per task, the increase is modest relative to the consistent gains in both partial progress and complete task resolution.



(a) Pass-rate comparison across different instruction-length groups.



(b) Pass-rate comparison across different agent interaction-length groups.

Figure 3: Pass-rate comparison among textual specifications, executable specifications, and RTADev across tasks grouped by instruction length and agent interaction turns.

RQ3: Executable vs. Textual Specifications

We further investigate whether executable specifications preserve design constraints more effectively than textual specifications. CODESPEC (Text) retains the same evidence-grounded functional-chain construction as CODESPEC, but represents the resulting design as textual information rather than executable specifications. We compare it with CODESPEC and RTADev, another code agent based on textual design planning. We compared the three methods described above on the FeatureBench Lite split.

As shown in Figure 3, the three methods perform relatively similarly on simpler tasks with instructions below 3,000 words or interactions under 120 turns. The gaps widen as task complexity increases: CODESPEC achieves 71.8% on instructions of 3,000–5,000 words, compared with 43.8% for CODESPEC (Text), and reaches 65.2% beyond 200 turns, outperforming the two textual approaches by 14.0 and 15.1 points. Although textual plans also provide useful design information, placing all details in the context increases input size and the risk of overlooking requirements and design constraints. In contrast, CODESPEC provides lightweight executable feedback throughout interaction, offering more reliable guidance for requirement-intensive and long-horizon tasks.

Agent Scaffold	Easy	Medium	Hard	Overall		
	%Passed	%Passed	%Passed	%Passed	#Resolved	Avg.Cost (\$)
OpenHands	48.6 $\pm$ 6.5	26.2 $\pm$ 4.4	9.4 $\pm$ 2.2	26.4 $\pm$ 2.9	2	0.10
Claude Code	62.0 $\pm$ 5.6	45.7 $\pm$ 4.6	19.3 $\pm$ 3.7	41.6 $\pm$ 3.2	5	0.11
Mini-SWE-Agent	58.0 $\pm$ 6.3	35.0 $\pm$ 4.2	18.7 $\pm$ 4.2	35.5 $\pm$ 3.1	4	0.08
RTADev	61.3 $\pm$ 6.3	47.4 $\pm$ 4.7	16.3 $\pm$ 4.8	41.3 $\pm$ 3.5	7	0.14
CODESPEC	70.0$\pm$5.6	51.7$\pm$4.3	20.4$\pm$4.7	46.6$\pm$3.3	8	0.17

Table 4: Comparison on NL2Repo using DeepSeek-V4-Pro (max thinking). Easy, Medium, and Hard correspond to repositories with <1.5K, 1.5K–4K, and >4K lines of code (LOC). Bold indicates the best result.

RQ4: Greenfield Repository Development Generalization

To assess generalization beyond repository evolution, we evaluate CODESPEC on NL2Repo-Bench (Ding et al. 2026), which contains 104 greenfield tasks that require generating a Python repository from a requirements document and an empty workspace. CODESPEC first creates a repository skeleton and functional units, after which the evolving codebase provides repository evidence for functional-chain construction and specification compilation.

As shown in Table 4, CODESPEC achieves 70.0%, 51.7%, and 20.4% %Passed on the Easy, Medium, and Hard splits, outperforming the strongest baselines by 8.0, 4.3, and 1.1 percentage points, respectively. It also obtains the best overall %Passed (46.6%) and #Resolved (8), demonstrating its generalizability to greenfield development. The larger gains on Easy and Medium tasks suggest that CODESPEC effectively organizes requirements into coherent implementations for repositories within approximately 4K lines of code, while the smaller gain on Hard tasks reflects the remaining difficulty of coordinating larger requirements for current agents. Although its additional specification and feedback steps slightly increase the average cost to 0.17, the performance gains indicate a favorable quality–cost trade-off.

Discussion

Case Study

Figure 4 presents a representative MLflow feature-development task in Featurebench that requires the agent to preserve a complete span lifecycle across multiple modules. The RTADev and CODESPEC (Text) identify the major entities, but fail to preserve several cross-module relations during implementation. In particular, it dispatches spans based on `is_recording()` rather than the required OTel span type, misses a required import between modules, and leaves the OTLP conversion path incomplete. Although many individual interfaces are implemented, these broken connections lead to an incomplete feature and a pass rate of 56.7%.

In contrast, CODESPEC compiles the functional chain into complementary executable architecture and behavior specifications. Architecture feedback reveals whether the chain remains complete and connected, while behavior feedback identifies whether the connected path realizes the intended behavior correctly. This lightweight, targeted feedback al-

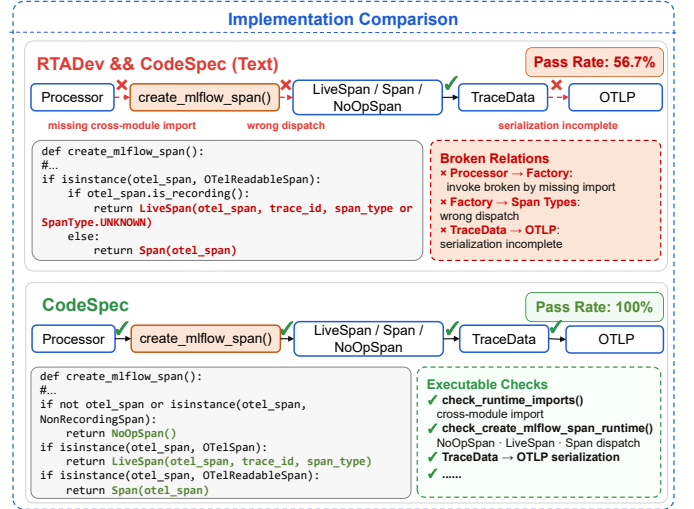


Figure 4: Case study of MLflow distributed tracing in Featurebench by using DeepSeek-V4-Pro.

lows the agent to localize and repair structural or behavioral violations during implementation. The case therefore shows that executable specifications go beyond textual design descriptions by continuously enforcing the functional chain and maintaining design–implementation consistency across long-horizon development.

Conclusion

We propose CODESPEC, a dual-specification approach for agentic long-horizon feature development. It constructs evidence-grounded functional chains from feature requirements and repository evidence, and compiles each chain into complementary executable architecture and behavior specifications. Architecture specifications preserve the integrity of feature realization paths, while behavior specifications verify runtime correctness along those paths. Together, they provide fine-grained and persistent feedback throughout implementation. Experiments on FeatureBench show that CODESPEC consistently outperforms representative coding agents, particularly on complex and long-horizon tasks. Moreover, results on NL2Repo-Bench further demonstrate its generalizability across repository-level development settings.

References

- Anthropic. 2026. Claude Code: An Agentic Coding Tool. <https://docs.anthropic.com/en/docs/claude-code/overview>. Accessed: 2026-07-11.
- Austin, J.; Odena, A.; Nye, M.; Bosma, M.; Michalewski, H.; Dohan, D.; Jiang, E.; Cai, C.; Terry, M.; Le, Q.; et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.
- Chen, M. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
- Chow, T. S. 1978. Testing software design modeled by finite-state machines. *IEEE transactions on software engineering*, (3): 178–187.
- Claessen, K.; and Hughes, J. 2000. QuickCheck: a lightweight tool for random testing of Haskell programs. In *Proceedings of the fifth ACM SIGPLAN international conference on Functional programming*, 268–279.
- Ding, J.; Long, S.; Pu, C.; Zhou, H.; Gao, H.; Gao, X.; He, C.; Hou, Y.; Hu, F.; Li, Z.; Shi, W.; Wang, Z.; Zan, D.; Zhang, C.; Zhang, X.; Chen, Q.; Cheng, X.; Deng, B.; Gu, Q.; Hua, K.; Lin, J.; Liu, P.; Li, M.; Pan, X.; Peng, Z.; Qin, Y.; Shan, Y.; Tan, Z.; Xie, W.; Wang, Z.; Yuan, Y.; Zhang, J.; Zhao, E.; Zhao, Y.; Zhu, H.; Zhu, L.; Zou, C.; Ding, M.; Jiao, J.; Liu, J.; Liu, M.; Liu, Q.; Tao, C.; Yang, J.; Yang, T.; Zhang, Z.; Chen, X.; Huang, W.; and Zhang, G. 2026. NL2Repo-Bench: Towards Long-Horizon Repository Generation Evaluation of Coding Agents. *arXiv:2512.12730*.
- Du, Y.; Cai, Y.; Zhou, Y.; Wang, C.; Qian, Y.; Pang, X.; Liu, Q.; Hu, Y.; and Chen, S. 2025. Swe-dev: Evaluating and training autonomous feature-driven software development. *arXiv preprint arXiv:2505.16975*.
- Hong, S.; Zhuge, M.; Chen, J.; Zheng, X.; Cheng, Y.; Wang, J.; Zhang, C.; Yau, S.; Lin, Z.; Zhou, L.; et al. 2024. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, volume 2024, 23247–23275.
- Jimenez, C. E.; Yang, J.; Wettig, A.; Yao, S.; Pei, K.; Press, O.; and Narasimhan, K. 2024. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, volume 2024, 54107–54157.
- Kildall, G. A. 1973. A unified approach to global program optimization. In *Proceedings of the 1st annual ACM SIGACT-SIGPLAN symposium on Principles of programming languages*, 194–206.
- Li, J.; Li, G.; Zhao, Y.; Li, Y.; Liu, H.; Zhu, H.; Wang, L.; Liu, K.; Fang, Z.; Wang, L.; Ding, J.; Zhang, X.; Zhu, Y.; Dong, Y.; Jin, Z.; Li, B.; Huang, F.; Li, Y.; Gu, B.; and Yang, M. 2024. DevEval: A Manually-Annotated Code Generation Benchmark Aligned with Real-World Code Repositories. In Ku, L.-W.; Martins, A.; and Srikumar, V., eds., *Findings of the Association for Computational Linguistics: ACL 2024*, 3603–3614. Bangkok, Thailand: Association for Computational Linguistics.
- Li, W.; Zhang, X.; Guo, Z.; Mao, S.; Luo, W.; Peng, G.; Huang, Y.; Wang, H.; and Li, S. 2025. Fea-bench: A benchmark for evaluating repository-level code generation for feature implementation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 17160–17176.
- Liu, J.; Wang, G.; Yang, R.; Zeng, J.; Zhao, M.; and Cai, Y. 2025. RTADev: Intention Aligned Multi-Agent Framework for Software Development. In Che, W.; Nabende, J.; Shutova, E.; and Pilehvar, M. T., eds., *Findings of the Association for Computational Linguistics: ACL 2025*, 1548–1581. Vienna, Austria: Association for Computational Linguistics. ISBN 979-8-89176-256-5.
- Murphy, G. C.; Notkin, D.; and Sullivan, K. J. 2001. Software reflexion models: Bridging the gap between design and implementation. *IEEE Transactions on Software Engineering*, 27(4): 364–380.
- OpenAI. 2025. Introducing Codex. <https://openai.com/index/introducing-codex/>. Accessed: 2026-07-11.
- Ostrand, T. J.; and Balcer, M. J. 1988. The category-partition method for specifying and generating functional tests. *Communications of the ACM*, 31(6): 676–686.
- Pruijt, L.; Köppe, C.; van der Werf, J. M.; and Brinkkemper, S. 2017. The accuracy of dependency analysis in static architecture compliance checking. *Software: practice and Experience*, 47(2): 273–309.
- Qian, C.; Liu, W.; Liu, H.; Chen, N.; Dang, Y.; Li, J.; Yang, C.; Chen, W.; Su, Y.; Cong, X.; et al. 2024. Chatdev: Communicative agents for software development. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*, 15174–15186.
- Wang, P.; Zhang, L.; Liu, F.; Shi, L.; Li, M.; Shen, B.; and Fu, A. 2025a. CodeIF-Bench: Evaluating Instruction-Following Capabilities of Large Language Models in Interactive Code Generation. *arXiv:2503.22688*.
- Wang, P.; Zhang, L.; Liu, F.; Tao, C.; and Zhu, Y. 2026. CodeMEM: AST-Guided Adaptive Memory for Repository-Level Iterative Code Generation. *arXiv preprint arXiv:2601.02868*.
- Wang, P.; Zhang, L.; Liu, F.; Zhu, Y.; Xu, W.; Shi, L.; Lian, X.; Li, M.; Shen, B.; and Fu, A. 2025b. EfficientEdit: Accelerating Code Editing via Edit-Oriented Speculative Decoding. *arXiv:2506.02780*.
- Wang, X.; Li, B.; Song, Y.; Xu, F. F.; Tang, X.; Zhuge, M.; Pan, J.; Song, Y.; Li, B.; Singh, J.; et al. 2025c. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations*, volume 2025, 65882–65919.
- Yang, J.; Jimenez, C.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K.; and Press, O. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37: 50528–50652.
- Zhou, Q.; Zhang, J.; Wang, H.; Hao, R.; Wang, J.; Han, M.; Yang, Y.; Wu, S.; Pan, F.; Fan, L.; et al. 2026. Featurebench: Benchmarking agentic coding for complex feature development. *arXiv preprint arXiv:2602.10975*.